LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3

ANDROID BASIC WITH KOTLIN

Oleh:

ERISCHA MARSELA **NIM. 2410817120022**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic With Kotlin Sederhana ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman II. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : ERISCHA MARSELA
NIM : 2410817120022

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan	Irham Maulani Abdul Gani, S.Kom.,
NIM. 2310817210027	M.Kom.
	NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR TABEL 4
DAFTAR GAMBAR 5
SOAL 1 6
A. SOURCE CODE 7
B. OUTPUT CODE 68
C. PEMBAHASAN 71
SOAL 2 ..... **Error! Bookmark not defined.**
A. PEMBAHASAN ..... **Error! Bookmark not defined.**
TAUTAN GIT 77
ADDITIONAL ..... **Error! Bookmark not defined.**

DAFTAR TABEL

Tabel 1. Source Code MainActivity Compose..... 7
Tabel 2. Source Code AnimeViewModel Compose 9
Tabel 3. Source Code AnimeData Compose..... 10
Tabel 4. Source Code AnimeCharacter Compose..... 15
Tabel 5. SourceCode NavGraph Compose 16
Tabel 6. Source Code Screeen Compose 19
Tabel 7. Source Code MainActivity XML..... 20
Tabel 8. Source Code AnimeAdapter XML..... 23
Tabel 9. Source Code AnimeRepository XML..... 26
Tabel 10. Source Code AnimeItem XML 34
Tabel 11. Source Code HomeFragment 35
Tabel 12. Source Code DetailFragment XML 42
Tabel 13. Source Code AnimeViewModel XML 46
Tabel 14. Source Code Activity_main.xml..... 47
Tabel 15. Source Code item_anime.xml 48
Tabel 16. Source Code fragment_home.xml..... 55
Tabel 17. Source Code fragment_detail.xml..... 57

DAFTAR GAMBAR

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
    a) Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
    b) Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding
9. Aplikasi Mendukung Multi-language (bahasa indonesia dan inggris)
10. Terdapat 1 halaman/screen untuk melakukan pergantian bahasa
11. Gunakan Icon dari library ‘material’ (material,material2,material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
12. Diatas Scrollable list, terdapat sebuah “Item Highlight” yang menggunakan Carousel atau LazyRow dengan ketentuan: Item bisa

bergeser secara finite atau infinite loop sebanyak jumlah item yang ada pada scrollable-list

A. SOURCE CODE

MainActivity.Kt Compose

Tabel 1. Source Code MainActivity Compose

```
1.  package com.example.clickanime
2.
3.  import android.content.Context
4.  import android.os.Bundle
5.  import androidx.activity.ComponentActivity
6.  import androidx.activity.compose.setContent
7.  import androidx.activity.enableEdgeToEdge
8.  import
    androidx.compose.foundation.layout.fillMaxSize
9.  import androidx.compose.material3.Surface
10. import androidx.compose.runtime.*
11. import androidx.compose.ui.Modifier
12. import
    androidx.navigation.compose.rememberNavControll
    er
13. import
    com.example.clickanime.navigation.NavGraph
14. import
    com.example.clickanime.ui.theme.AnimeAppTheme
15. import java.util.Locale
16.
17. class MainActivity : ComponentActivity() {
18.
19.     override   fun   attachBaseContext(newBase:
    Context) {
```

```
20.         val prefs = newBase.getSharedPreferences("settings", Context.MODE_PRIVATE)
21.         val lang = prefs.getString("language", Locale.getDefault().language) ?: "en"
22.         val locale = Locale(lang)
23.         Locale.setDefault(locale)
24.         val config = newBase.resources.configuration
25.         config.setLocale(locale)
26.         super.attachBaseContext(newBase.createConfigurationContext(config))
27.     }
28.
29.     override fun onCreate(savedInstanceState: Bundle?) {
30.         super.onCreate(savedInstanceState)
31.         enableEdgeToEdge()
32.         setContent {
33.             AnimeAppTheme {
34.                 Surface(modifier = Modifier.fillMaxSize()) {
35.                     val navController = rememberNavController()
36.                     NavGraph(
37.                         navController = navController,
38.                         onLocaleChange = { langCode ->
39.                             applyLocale(langCode)
40.                         }
```

```
41.                         )
42.                     }
43.                 }
44.             }
45.         }
46.
47.         private fun applyLocale(langCode: String) {
48.             getSharedPreferences("settings",
    Context.MODE_PRIVATE)
49.                 .edit()
50.                 .putString("language", langCode)
51.                 .apply()
52.             recreate()
53.         }
54.     }
```

Tabel 2. Source Code AnimeViewModel Compose

```
1.  package com.example.clickanime.viewmodel
2.
3.  import androidx.lifecycle.ViewModel
4.  import com.example.clickanime.data.AnimeData
5.  import
    com.example.clickanime.model.AnimeCharacter
6.  import kotlinx.coroutines.flow.MutableStateFlow
7.  import kotlinx.coroutines.flow.StateFlow
8.  import kotlinx.coroutines.flow.asStateFlow
9.
10. class AnimeViewModel : ViewModel() {
```

```
11.
12.     private val _characters = MutableStateFlow<List<AnimeCharacter>>(AnimeData.characters)
13.     val characters: StateFlow<List<AnimeCharacter>> = _characters.asStateFlow()
14.
15.     private val _carouselIndex = MutableStateFlow(0)
16.     val carouselIndex: StateFlow<Int> = _carouselIndex.asStateFlow()
17.
18.     fun updateCarouselIndex(index: Int) {
19.         _carouselIndex.value = index
20.     }
21. }
```

Tabel 3. Source Code AnimeData Compose

```
1.  package com.example.clickanime.data
2.
3.  import com.example.clickanime.R
4.  import com.example.clickanime.model.AnimeCharacter
5.
6.  object AnimeData {
```

```
7.      val characters = listOf(
8.          AnimeCharacter(
9.              id = 1,
10.             nameResId = R.string.char1_name,
11.             animeResId = R.string.char1_anime,
12.             genreResId = R.string.char1_genre,
13.             descriptionResId = R.string.char1_description,
14.             abilityResId = R.string.char1_ability,
15.             imageUrl = "https://cdn.myanimelist.net/images/characters/2/284121.jpg",
16.             myAnimeListUrl = "https://myanimelist.net/character/17/Naruto_Uzumaki",
17.             rating = 9.2f,
18.             episodesOrChapters = 720,
19.             year = 2002,
20.             typeResId = R.string.type_anime
21.         ),
22.         AnimeCharacter(
23.             id = 2,
24.             nameResId = R.string.char2_name,
25.             animeResId = R.string.char2_anime,
```

```
            genreResId = R.string.char2_genre,
            descriptionResId = R.string.char2_description,
            abilityResId = R.string.char2_ability,
            imageUrl = "https://cdn.myanimelist.net/images/characters/9/310307.jpg",
            myAnimeListUrl = "https://myanimelist.net/character/40/Monkey_D_Luffy",
            rating = 9.4f,
            episodesOrChapters = 1100,
            year = 1999,
            typeResId = R.string.type_anime
        ),
        AnimeCharacter(
            id = 3,
            nameResId = R.string.char3_name,
            animeResId = R.string.char3_anime,
            genreResId = R.string.char3_genre,
            descriptionResId = R.string.char3_description,
            abilityResId = R.string.char3_ability,
```

```
43.             imageUrl                                =
"https://myanimelist.net/images/characters/8/349229.jpg",
44.             myAnimeListUrl                          =
"https://myanimelist.net/character/5/Ichigo_Kurosaki",
45.             rating = 8.9f,
46.             episodesOrChapters = 366,
47.             year = 2004,
48.             typeResId = R.string.type_anime
49.         ),
50.         AnimeCharacter(
51.             id = 4,
52.             nameResId = R.string.char4_name,
53.             animeResId = R.string.char4_anime,
54.             genreResId = R.string.char4_genre,
55.             descriptionResId                        =
R.string.char4_description,
56.             abilityResId                            =
R.string.char4_ability,
57.             imageUrl                                =
"https://myanimelist.net/images/characters/9/206427.jpg",
58.             myAnimeListUrl                          =
"https://myanimelist.net/character/40882/Eren_Yeager",
```

```
59.             rating = 9.1f,
60.             episodesOrChapters = 87,
61.             year = 2013,
62.             typeResId = R.string.type_anime
63.         ),
64.         AnimeCharacter(
65.             id = 5,
66.             nameResId = R.string.char5_name,
67.             animeResId = R.string.char5_anime,
68.             genreResId = R.string.char5_genre,
69.             descriptionResId                    =
    R.string.char5_description,
70.             abilityResId                        =
    R.string.char5_ability,
71.             imageUrl                            =
    "https://myanimelist.net/images/characters/15/7
    2546.jpg",
72.             myAnimeListUrl                      =
    "https://myanimelist.net/character/246/Goku",
73.             rating = 9.0f,
74.             episodesOrChapters = 291,
75.             year = 1989,
76.             typeResId = R.string.type_anime
77.         ),
78.         AnimeCharacter(
```

```
79.             id = 6,
80.             nameResId = R.string.char6_name,
81.             animeResId = R.string.char6_anime,
82.             genreResId = R.string.char6_genre,
83.             descriptionResId = R.string.char6_description,
84.             abilityResId = R.string.char6_ability,
85.             imageUrl = "https://myanimelist.net/images/characters/9/72533.jpg",
86.             myAnimeListUrl = "https://myanimelist.net/character/11/Edward_Elric",
87.             rating = 9.3f,
88.             episodesOrChapters = 64,
89.             year = 2009,
90.             typeResId = R.string.type_anime
91.         )
92.     )
93. }
```

Tabel 4. Source Code AnimeCharacter Compose

```
1.  package com.example.clickanime.model
2.
3.  data class AnimeCharacter(
```

```
4.      val id: Int,
5.      val nameResId: Int,
6.      val animeResId: Int,
7.      val genreResId: Int,
8.      val descriptionResId: Int,
9.      val abilityResId: Int,
10.     val imageUrl: String,
11.     val myAnimeListUrl: String,
12.     val rating: Float,
13.     val episodesOrChapters: Int,
14.     val year: Int,
15.     val typeResId: Int
16. )
```

Tabel 5. SourceCode NavGraph Compose

```
1.  package com.example.clickanime.navigation
2.
3.  import androidx.compose.runtime.Composable
4.  import androidx.navigation.NavHostController
5.  import androidx.navigation.NavType
6.  import androidx.navigation.compose.NavHost
7.  import androidx.navigation.compose.composable
8.  import androidx.navigation.navArgument
9.  import com.example.clickanime.data.AnimeData
```

```
10. import
    com.example.clickanime.ui.detail.DetailScreen
11. import
    com.example.clickanime.ui.home.HomeScreen
12. import
    com.example.clickanime.ui.language.LanguageScre
    en
13.
14. @Composable
15. fun NavGraph(
16.     navController: NavHostController,
17.     onLocaleChange: (String) -> Unit
18. ) {
19.     NavHost(
20.         navController = navController,
21.         startDestination = Screen.Home.route
22.     ) {
23.         composable(Screen.Home.route) {
24.             HomeScreen(
25.                 characters                    =
    AnimeData.characters,
26.                 onDetailClick = { characterId -
    >
```

```
27.
navController.navigate(Screen.Detail.createRoute(characterId))
28.                 },
29.                 onLanguageClick = {
30.
navController.navigate(Screen.Language.route)
31.                 }
32.             )
33.         }
34.         composable(
35.             route = Screen.Detail.route,
36.             arguments = listOf(navArgument("characterId") { type = NavType.IntType })
37.         ) { backStackEntry ->
38.             val characterId = backStackEntry.arguments?.getInt("characterId") ?: return@composable
39.             val character = AnimeData.characters.find { it.id == characterId }
40.             character?.let {
41.                 DetailScreen(
42.                     character = it,
```

```
43.                     onBack          =          {
    navController.popBackStack() }
44.                 )
45.             }
46.         }
47.         composable(Screen.Language.route) {
48.             LanguageScreen(
49.                 onLanguageSelected = { langCode
    ->
50.                     onLocaleChange(langCode)
51.
    navController.popBackStack()
52.                 },
53.                 onBack          =          {
    navController.popBackStack() }
54.             )
55.         }
56.     }
57. }
```

Tabel 6. Source Code Screeen Compose

```
1.  package com.example.clickanime.navigation
2.
3.  sealed class Screen(val route: String) {
4.      object Home : Screen("home")
```

```
5.      object Detail : Screen("detail/{characterId}") {
6.          fun createRoute(characterId: Int) = "detail/$characterId"
7.      }
8.      object Language : Screen("language")
9.  }
```

MainActivity.Kt XML

Tabel 7. Source Code MainActivity XML

```
1.  package com.example.animepopular
2.
3.  import android.content.Context
4.  import android.os.Bundle
5.  import androidx.appcompat.app.AppCompatActivity
6.  import androidx.navigation.fragment.NavHostFragment
7.  import androidx.navigation.ui.setupActionBarWithNavController
8.  import com.example.animepopular.databinding.ActivityMainBinding
9.  import java.util.Locale
10.
```

```
11. class MainActivity : AppCompatActivity() {
12.
13.     private lateinit var binding: ActivityMainBinding
14.
15.     override fun attachBaseContext(newBase: Context) {
16.         val prefs = newBase.getSharedPreferences("settings", Context.MODE_PRIVATE)
17.         val lang = prefs.getString("language", "en") ?: "en"
18.         val locale = Locale(lang)
19.         Locale.setDefault(locale)
20.         val config = newBase.resources.configuration
21.         config.setLocale(locale)
22.         val context = newBase.createConfigurationContext(config)
23.         super.attachBaseContext(context)
24.     }
25.
26.     override fun onCreate(savedInstanceState: Bundle?) {
27.         super.onCreate(savedInstanceState)
```

```
28.         binding                                          =
    ActivityMainBinding.inflate(layoutInflater)
29.         setContentView(binding.root)
30.
31.         val           navHostFragment           =
    supportFragmentManager
32.
    .findFragmentById(R.id.nav_host_fragment)      as
    NavHostFragment
33.         val            navController             =
    navHostFragment.navController
34.
35.         setSupportActionBar(binding.toolbar)
36.
    setupActionBarWithNavController(navController)
37.
38.
    navController.addOnDestinationChangedListener   {
    _, destination, _ ->
39.             supportActionBar?.title              =
    destination.label
40.         }
41.     }
42.
43.     override fun onSupportNavigateUp(): Boolean {
```

```
44.         val navHostFragment =
    supportFragmentManager
45.
    .findFragmentById(R.id.nav_host_fragment) as
    NavHostFragment
46.         return
    navHostFragment.navController.navigateUp() ||
    super.onSupportNavigateUp()
47.     }
48. }
```

Tabel 8. Source Code AnimeAdapter XML

```
1.  package com.example.animepopular.adapter
2.
3.  import android.content.Context
4.  import android.view.LayoutInflater
5.  import android.view.ViewGroup
6.  import androidx.recyclerview.widget.DiffUtil
7.  import androidx.recyclerview.widget.ListAdapter
8.  import androidx.recyclerview.widget.RecyclerView
9.  import com.bumptech.glide.Glide
10. import
    com.example.animepopular.databinding.ItemAnimeBin
    ding
11. import com.example.animepopular.model.AnimeItem
12.
```

```
13. class AnimeAdapter(
14.     private val onImdbClick: (AnimeItem) -> Unit,
15.     private val onDetailClick: (AnimeItem) -> Unit
16. ) : ListAdapter<AnimeItem, AnimeAdapter.AnimeViewHolder>(DiffCallback()) {
17. 
18.     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): AnimeViewHolder {
19.         val binding = ItemAnimeBinding.inflate(
20.             LayoutInflater.from(parent.context), parent, false
21.         )
22.         return AnimeViewHolder(binding, parent.context)
23.     }
24. 
25.     override fun onBindViewHolder(holder: AnimeViewHolder, position: Int) {
26.         holder.bind(getItem(position))
27.     }
28. 
29.     inner class AnimeViewHolder(
30.         private val binding: ItemAnimeBinding,
31.         private val context: Context
32.     ) : RecyclerView.ViewHolder(binding.root) {
```

```
33.
34.         fun bind(anime: AnimeItem) {
35.             val              prefs              =
    context.getSharedPreferences("settings",
    Context.MODE_PRIVATE)
36.             val               isId              =
    prefs.getString("language", "en") == "id"
37.
38.             binding.tvTitle.text   =   if   (isId)
    anime.titleId else anime.titleEn
39.             binding.tvYear.text                  =
    anime.year.toString()
40.             binding.tvGenre.text   =   if   (isId)
    anime.genreId else anime.genreEn
41.             binding.tvPlot.text   =   if   (isId)
    anime.plotId else anime.plotEn
42.             binding.tvRating.text = anime.rating
43.             binding.tvEpisodes.text              =
    anime.episodes
44.
45.             Glide.with(context)
46.                 .load(anime.imageUrl)
47.
    .placeholder(android.R.drawable.ic_menu_gallery)
48.
    .error(android.R.drawable.ic_menu_gallery)
```

```
49.                 .into(binding.imgPoster)
50.
51.             binding.btnImdb.setOnClickListener { onImdbClick(anime) }
52.             binding.btnDetail.setOnClickListener { onDetailClick(anime) }
53.         }
54.     }
55.
56.     class DiffCallback : DiffUtil.ItemCallback<AnimeItem>() {
57.         override fun areItemsTheSame(oldItem: AnimeItem, newItem: AnimeItem) = oldItem.id == newItem.id
58.         override fun areContentsTheSame(oldItem: AnimeItem, newItem: AnimeItem) = oldItem == newItem
59.     }
60. }
```

Tabel 9. Source Code AnimeRepository XML

```
1. package com.example.animepopular.data
2.
3. import com.example.animepopular.model.AnimeItem
4.
5. object AnimeRepository {
```

```
6.
7.      fun getAnimeList(): List<AnimeItem> = listOf(
8.          AnimeItem(
9.              id = 1,
10.             titleEn = "Attack on Titan",
11.             titleId = "Serangan Titan",
12.             year = 2013,
13.             genreEn = "Action / Dark Fantasy",
14.             genreId = "Aksi / Fantasi Gelap",
15.             plotEn = "In a world where humanity
    lives within enormous walled cities to protect
    themselves from Titans, gigantic humanoid
    creatures, young Eren Yeager vows to exterminate
    all Titans after his mother is consumed by one.",
16.             plotId = "Di dunia di mana umat
    manusia hidup dalam kota berdinding raksasa untuk
    melindungi diri dari Titan, makhluk humanoid
    raksasa, Eren Yeager yang muda bersumpah untuk
    memusnahkan semua Titan setelah ibunya dimakan
    oleh salah satunya.",
17.             studioEn = "Wit Studio / MAPPA",
18.             studioId = "Wit Studio / MAPPA",
19.             episodes = "87 Episodes",
20.             rating = "★ 9.0 / 10",
```

```
21.             imageUrl                                =
    "https://cdn.myanimelist.net/images/anime/10/473
    47.jpg",
22.             imdbUrl                                 =
    "https://www.imdb.com/title/tt2560140/"
23.         ),
24.         AnimeItem(
25.             id = 2,
26.             titleEn = "Hunter x Hunter",
27.             titleId = "Hunter x Hunter",
28.             year = 2011,
29.             genreEn = "Adventure / Fantasy",
30.             genreId = "Petualangan / Fantasi",
31.             plotEn = "Gon Freecss aspires to
    become a Hunter, a licensed professional who
    specializes in tracking down fantastical
    treasures, rare animals, or even other people, to
    find his missing father who was also a legendary
    Hunter.",
32.             plotId = "Gon Freecss bercita-cita
    menjadi Hunter, seorang profesional berlisensi
    yang mengkhususkan diri dalam melacak harta
    ajaib, hewan langka, bahkan orang lain, untuk
    menemukan ayahnya yang hilang yang juga seorang
    Hunter legendaris.",
33.             studioEn = "Madhouse",
34.             studioId = "Madhouse",
```

```
35.             episodes = "148 Episodes",
36.             rating = "★ 9.0 / 10",
37.             imageUrl                                =
"https://cdn.myanimelist.net/images/anime/11/336
57.jpg",
38.             imdbUrl                                 =
"https://www.imdb.com/title/tt2098220/"
39.         ),
40.         AnimeItem(
41.             id = 3,
42.             titleEn = "Gurren Lagann",
43.             titleId = "Gurren Lagann",
44.             year = 2007,
45.             genreEn = "Mecha / Action",
46.             genreId = "Mecha / Aksi",
47.             plotEn = "Simon and Kamina were born
and raised in a deep underground village. One day
they find a small mecha which helps them break
through the surface and discover the vast world
above, where they must fight against the Spiral
King.",
48.             plotId = "Simon dan Kamina lahir dan
dibesarkan di desa bawah tanah. Suatu hari mereka
menemukan mecha kecil yang membantu mereka
menembus permukaan dan menemukan dunia luas di
atas, di mana mereka harus melawan Raja Spiral.",
```

```
49.             studioEn = "Gainax",
50.             studioId = "Gainax",
51.             episodes = "27 Episodes",
52.             rating = "★ 8.7 / 10",
53.             imageUrl                                =
    "https://myanimelist.net/images/anime/4/5123l.jpg",
54.             imdbUrl                                 =
    "https://www.imdb.com/title/tt0948103/"
55.         ),
56.         AnimeItem(
57.             id = 4,
58.             titleEn = "Code Geass",
59.             titleId = "Code Geass",
60.             year = 2008,
61.             genreEn = "Mecha / Drama",
62.             genreId = "Mecha / Drama",
63.             plotEn = "A disgraced prince uses a
    power called Geass to lead a rebellion against
    the Holy Britannian Empire that has conquered
    Japan, while hiding his identity as the masked
    revolutionary Zero.",
64.             plotId = "Seorang pangeran yang
    dipermalukan menggunakan kekuatan bernama Geass
    untuk memimpin pemberontakan melawan Kekaisaran
    Britannia Suci yang telah menaklukkan Jepang,
```

```
        sambil menyembunyikan identitasnya sebagai
        revolusioner bertopeng Zero.",
65.             studioEn = "Sunrise",
66.             studioId = "Sunrise",
67.             episodes = "50 Episodes",
68.             rating = "★ 8.7 / 10",
69.             imageUrl                                =
        "https://myanimelist.net/images/anime/1032/135088l.jpg",
70.             imdbUrl                                 =
        "https://www.imdb.com/title/tt0994314/"
71.         ),
72.         AnimeItem(
73.             id = 5,
74.             titleEn = "Demon Slayer",
75.             titleId = "Pembasmi Iblis",
76.             year = 2019,
77.             genreEn = "Action / Supernatural",
78.             genreId = "Aksi / Supranatural",
79.             plotEn = "Tanjiro Kamado becomes a
        Demon Slayer of the Demon Slayer Corps after his
        family was slaughtered and his younger sister
        Nezuko turned into a demon, to avenge their family
        and cure his sister.",
80.             plotId = "Tanjiro Kamado menjadi
        Pembasmi Iblis setelah keluarganya dibantai dan
```

```
        adiknya Nezuko berubah menjadi iblis, untuk membalas dendam keluarga dan menyembuhkan adiknya.",
81.             studioEn = "ufotable",
82.             studioId = "ufotable",
83.             episodes = "44+ Episodes",
84.             rating = "★ 8.7 / 10",
85.             imageUrl = "https://myanimelist.net/images/anime/1286/99889l.jpg",
86.             imdbUrl = "https://www.imdb.com/title/tt9335498/"
87.         ),
88.         AnimeItem(
89.             id = 6,
90.             titleEn = "Trigun",
91.             titleId = "Trigun",
92.             year = 1998,
93.             genreEn = "Sci-Fi / Western",
94.             genreId = "Fiksi Ilmiah / Barat",
95.             plotEn = "Vash the Stampede is a legendary gunman with a bounty of 60 billion double dollars on his head. He roams the desert planet Gunsmoke, followed by two insurance agents, though he is actually a pacifist.",
```

```
96.             plotId = "Vash the Stampede adalah
penembak legendaris dengan hadiah 60 miliar
double dollar di kepalanya. Ia berkeliaran di
planet gurun Gunsmoke, diikuti dua agen asuransi,
meskipun sebenarnya ia seorang pasifis.",
97.             studioEn = "Madhouse",
98.             studioId = "Madhouse",
99.             episodes = "26 Episodes",
100.            rating = "★ 8.2 / 10",
101.            imageUrl                          =
"https://myanimelist.net/images/anime/7/20310l.j
pg",
102.            imdbUrl                           =
"https://www.imdb.com/title/tt0251439/"
103.        ),
104.        AnimeItem(
105.            id = 7,
106.            titleEn = "Spy x Family",
107.            titleId = "Spy x Family",
108.            year = 2022,
109.            genreEn = "Comedy / Action",
110.            genreId = "Komedi / Aksi",
111.            plotEn = "A spy on a secret mission
must build a fake family to complete it. He
recruits a telepathic girl as his daughter and an
```

```
        assassin as his wife, neither knowing each other's
        true identities.",
112.            plotId = "Seorang mata-mata dalam
        misi rahasia harus membangun keluarga palsu untuk
        menyelesaikannya. Ia merekrut gadis telepati
        sebagai putrinya dan seorang pembunuh bayaran
        sebagai istrinya, tanpa mengetahui identitas asli
        satu sama lain.",
113.            studioEn    =    "Wit    Studio    /
        CloverWorks",
114.            studioId    =    "Wit    Studio    /
        CloverWorks",
115.            episodes = "37 Episodes",
116.            rating = "★ 8.5 / 10",
117.            imageUrl                              =
        "https://myanimelist.net/images/anime/1111/12126
        2l.jpg",
118.            imdbUrl                               =
        "https://www.imdb.com/title/tt13706018/"
119.        )
120.    )
121. }
```

Tabel 10. Source Code AnimeItem XML

```
1.  package com.example.animepopular.model
2.
3.  import android.os.Parcelable
```

```
4.  import kotlinx.parcelize.Parcelize
5.
6.  @Parcelize
7.  data class AnimeItem(
8.      val id: Int,
9.      val titleEn: String,
10.     val titleId: String,
11.     val year: Int,
12.     val genreEn: String,
13.     val genreId: String,
14.     val plotEn: String,
15.     val plotId: String,
16.     val studioEn: String,
17.     val studioId: String,
18.     val episodes: String,
19.     val rating: String,
20.     val imageUrl: String,
21.     val imdbUrl: String
22. ) : Parcelable
```

Tabel 11. Source Code HomeFragment

```
1.  package com.example.animepopular.ui.home
2.
3.  import android.content.Intent
```

```
4.  import android.net.Uri
5.  import android.os.Bundle
6.  import android.view.LayoutInflater
7.  import android.view.Menu
8.  import android.view.MenuInflater
9.  import android.view.MenuItem
10. import android.view.View
11. import android.view.ViewGroup
12. import androidx.core.view.MenuProvider
13. import androidx.fragment.app.Fragment
14. import androidx.fragment.app.viewModels
15. import androidx.lifecycle.Lifecycle
16. import
    androidx.navigation.fragment.findNavController
17. import
    androidx.recyclerview.widget.GridLayoutManager
18. import
    androidx.recyclerview.widget.LinearLayoutManager
19. import com.example.animepopular.R
20. import
    com.example.animepopular.adapter.AnimeAdapter
21. import
    com.example.animepopular.adapter.CarouselAdapter
```

```
22. import com.example.animepopular.databinding.FragmentHomeBinding
23. import com.example.animepopular.model.AnimeItem
24. import com.example.animepopular.viewmodel.AnimeViewModel
25.
26. class HomeFragment : Fragment() {
27.
28.     private var _binding: FragmentHomeBinding? = null
29.     private val binding get() = _binding!!
30.     private val viewModel: AnimeViewModel by viewModels()
31.
32.     private lateinit var animeAdapter: AnimeAdapter
33.     private lateinit var carouselAdapter: CarouselAdapter
34.
35.     override fun onCreateView(
36.         inflater: LayoutInflater, container: ViewGroup?,
37.         savedInstanceState: Bundle?
38.     ): View {
```

```
39.         _binding = FragmentHomeBinding.inflate(inflater, container, false)
40.         return binding.root
41.     }
42.
43.     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
44.         super.onViewCreated(view, savedInstanceState)
45.
46.         setupMenu()
47.         setupCarousel()
48.         setupRecyclerView()
49.         observeData()
50.     }
51.
52.     private fun setupMenu() {
53.         requireActivity().addMenuProvider(object : MenuProvider {
54.             override fun onCreateMenu(menu: Menu, menuInflater: MenuInflater) {
55.
menuInflater.inflate(R.menu.home_menu, menu)
56.             }
```

```
57.             override                                fun
    onMenuItemSelected(menuItem: MenuItem): Boolean {
58.                 return when (menuItem.itemId) {
59.                     R.id.action_language -> {
60.
    findNavController().navigate(R.id.action_homeFra
    gment_to_languageFragment)
61.                         true
62.                     }
63.                     else -> false
64.                 }
65.             }
66.         },                    viewLifecycleOwner,
    Lifecycle.State.RESUMED)
67.     }
68.
69.     private fun setupCarousel() {
70.         carouselAdapter = CarouselAdapter(
71.             onImdbClick    =    {    anime    ->
    openUrl(anime.imdbUrl) },
72.             onDetailClick    =    {    anime    ->
    navigateToDetail(anime) }
73.         )
74.         binding.viewPagerCarousel.adapter      =
    carouselAdapter
```

```
75.
76.         // Attach dots indicator
77.
    binding.dotsIndicator.attachTo(binding.viewPager
    Carousel)
78.     }
79.
80.     private fun setupRecyclerView() {
81.         animeAdapter = AnimeAdapter(
82.             onImdbClick    =    {    anime    ->
    openUrl(anime.imdbUrl) },
83.             onDetailClick    =    {    anime    ->
    navigateToDetail(anime) }
84.         )
85.
86.         val       layoutManager       =       if
    (resources.configuration.orientation ==
87.
    android.content.res.Configuration.ORIENTATION_LA
    NDSCAPE) {
88.             GridLayoutManager(requireContext(),
    2)
89.         } else {
90.
    LinearLayoutManager(requireContext())
91.         }
```

```
92.         binding.rvAnime.layoutManager =
    layoutManager
93.         binding.rvAnime.adapter = animeAdapter
94.     }
95.
96.     private fun observeData() {
97.
    viewModel.animeList.observe(viewLifecycleOwner)
    { list ->
98.             animeAdapter.submitList(list)
99.             carouselAdapter.submitList(list)
100.        }
101.    }
102.
103.    private fun openUrl(url: String) {
104.        val intent = Intent(Intent.ACTION_VIEW,
    Uri.parse(url))
105.        startActivity(intent)
106.    }
107.
108.    private fun navigateToDetail(anime:
    AnimeItem) {
109.        val action =
    HomeFragmentDirections.actionHomeFragmentToDetai
    lFragment(anime)
```

```
110.            findNavController().navigate(action)
111.        }
112.
113.        override fun onDestroyView() {
114.            super.onDestroyView()
115.            _binding = null
116.        }
117.    }
```

Tabel 12. Source Code DetailFragment XML

```
1.  package com.example.animepopular.ui.detail
2.
3.  import android.content.Context
4.  import android.content.Intent
5.  import android.net.Uri
6.  import android.os.Bundle
7.  import android.view.LayoutInflater
8.  import android.view.View
9.  import android.view.ViewGroup
10. import androidx.fragment.app.Fragment
11. import com.bumptech.glide.Glide
12. import
    com.example.animepopular.databinding.FragmentDet
    ailBinding
13. import com.example.animepopular.model.AnimeItem
```

```
14.
15. class DetailFragment : Fragment() {
16.
17.     private var _binding: FragmentDetailBinding?
    = null
18.     private val binding get() = _binding!!
19.
20.     override fun onCreateView(
21.         inflater:   LayoutInflater,   container:
    ViewGroup?,
22.         savedInstanceState: Bundle?
23.     ): View {
24.         _binding                                =
    FragmentDetailBinding.inflate(inflater,
    container, false)
25.         return binding.root
26.     }
27.
28.     override   fun   onViewCreated(view:   View,
    savedInstanceState: Bundle?) {
29.         super.onViewCreated(view,
    savedInstanceState)
30.
```

```
31.         val anime =
DetailFragmentArgs.fromBundle(requireArguments()
).anime
32.         val isIndonesian = getCurrentLanguage()
== "id"
33.         bindData(anime, isIndonesian)
34.     }
35.
36.     private fun getCurrentLanguage(): String {
37.         val prefs =
requireContext().getSharedPreferences("settings"
, Context.MODE_PRIVATE)
38.         return prefs.getString("language", "en")
?: "en"
39.     }
40.
41.     private fun bindData(anime: AnimeItem,
isIndonesian: Boolean) {
42.         Glide.with(this)
43.             .load(anime.imageUrl)
44.
.placeholder(android.R.drawable.ic_menu_gallery)
45.
.error(android.R.drawable.ic_menu_gallery)
46.             .centerCrop()
47.             .into(binding.imgDetailPoster)
```

```
48.
49.         binding.tvDetailTitle.text      =      if
(isIndonesian) anime.titleId else anime.titleEn
50.         binding.tvDetailYear.text               =
anime.year.toString()
51.         binding.tvDetailGenre.text      =      if
(isIndonesian) anime.genreId else anime.genreEn
52.         binding.tvDetailStudio.text     =     if
(isIndonesian) anime.studioId else anime.studioEn
53.         binding.tvDetailEpisodes.text           =
anime.episodes
54.         binding.tvDetailRating.text             =
anime.rating
55.         binding.tvDetailPlot.text      =      if
(isIndonesian) anime.plotId else anime.plotEn
56.
57.         binding.btnDetailImdb.setOnClickListener
{
58.             val             intent             =
Intent(Intent.ACTION_VIEW,
Uri.parse(anime.imdbUrl))
59.             startActivity(intent)
60.         }
61.     }
62.
63.     override fun onDestroyView() {
```

```
64.         super.onDestroyView()
65.         _binding = null
66.     }
67. }
```

Tabel 13. Source Code AnimeViewModel XML

```
1.  package com.example.animepopular.viewmodel
2.
3.  import androidx.lifecycle.LiveData
4.  import androidx.lifecycle.MutableLiveData
5.  import androidx.lifecycle.ViewModel
6.  import
    com.example.animepopular.data.AnimeRepository
7.  import com.example.animepopular.model.AnimeItem
8.
9.  class AnimeViewModel : ViewModel() {
10.
11.     private val _animeList =
    MutableLiveData<List<AnimeItem>>()
12.     val animeList: LiveData<List<AnimeItem>> =
    _animeList
13.
14.     init {
15.         _animeList.value =
    AnimeRepository.getAnimeList()
16.     }
```

```
17.
18.     fun    getAnimeList():    List<AnimeItem>    =
AnimeRepository.getAnimeList()
19. }
```

Activity_main.xml

Tabel 14. Source Code Activity_main.xml

```
1.  <?xml version="1.0" encoding="utf-8"?>
2.  <androidx.coordinatorlayout.widget.CoordinatorL
    ayout
3.
    xmlns:android="http://schemas.android.com/apk/r
    es/android"
4.
    xmlns:app="http://schemas.android.com/apk/res-
    auto"
5.      android:layout_width="match_parent"
6.      android:layout_height="match_parent">
7.
8.
    <com.google.android.material.appbar.AppBarLayou
    t
9.          android:id="@+id/appBarLayout"
10.         android:layout_width="match_parent"
11.         android:layout_height="wrap_content"
12.
    android:theme="@style/ThemeOverlay.MaterialComp
    onents.Dark.ActionBar">
13.
14.         <androidx.appcompat.widget.Toolbar
15.             android:id="@+id/toolbar"
```

```
16.             android:layout_width="match_parent"
17.
android:layout_height="?attr/actionBarSize"
18.
android:background="?attr/colorPrimary"
19.
app:popupTheme="@style/ThemeOverlay.MaterialCom
ponents.Light"
20.             app:title="@string/app_name" />
21.
22.
</com.google.android.material.appbar.AppBarLayo
ut>
23.
24.
<androidx.fragment.app.FragmentContainerView
25.         android:id="@+id/nav_host_fragment"
26.
android:name="androidx.navigation.fragment.NavH
ostFragment"
27.         android:layout_width="match_parent"
28.         android:layout_height="match_parent"
29.
app:layout_behavior="@string/appbar_scrolling_v
iew_behavior"
30.         app:defaultNavHost="true"
31.         app:navGraph="@navigation/nav_graph" />
32.
33. </androidx.coordinatorlayout.widget.Coordinator
    Layout>
```

Tabel 15. Source Code item_anime.xml

```
1.  <?xml version="1.0" encoding="utf-8"?>
2.  <com.google.android.material.card.MaterialCardV
    iew
```

```
3.  xmlns:android="http://schemas.android.com/apk/res/android"
4.  xmlns:app="http://schemas.android.com/apk/res-auto"
5.      android:layout_width="match_parent"
6.      android:layout_height="wrap_content"
7.      android:layout_marginBottom="12dp"
8.  app:cardBackgroundColor="@color/card_background"
9.      app:cardCornerRadius="16dp"
10.     app:cardElevation="4dp">
11.
12.     <LinearLayout
13.         android:layout_width="match_parent"
14.         android:layout_height="wrap_content"
15.         android:orientation="horizontal"
16.         android:padding="12dp">
17.
18.
19. <com.google.android.material.imageview.ShapeableImageView
20.             android:id="@+id/imgPoster"
21.             android:layout_width="100dp"
22.             android:layout_height="140dp"
23.             android:scaleType="centerCrop"
24. app:shapeAppearanceOverlay="@style/RoundedCornerImage" />
25.
```

```
26.
27.         <LinearLayout
28.             android:layout_width="0dp"
29.             android:layout_height="wrap_content"
30.             android:layout_weight="1"
31.             android:orientation="vertical"
32.             android:paddingStart="12dp">
33.
34.
35.             <LinearLayout
36.                 android:layout_width="match_parent"
37.                 android:layout_height="wrap_content"
38.                 android:orientation="horizontal"
39.                 android:gravity="center_vertical">
40.
41.                 <TextView
42.                     android:id="@+id/tvTitle"
43.                     android:layout_width="0dp"
44.                     android:layout_height="wrap_content"
45.                     android:layout_weight="1"
46.                     android:textColor="@color/text_primary"
47.                     android:textSize="16sp"
48.                     android:textStyle="bold"
49.                     android:maxLines="2"
50.                     android:ellipsize="end" />
```

```
51.
52.                 <TextView
53.                     android:id="@+id/tvYear"
54.
    android:layout_width="wrap_content"
55.
    android:layout_height="wrap_content"
56.
    android:textColor="@color/text_secondary"
57.                     android:textSize="12sp"
58.                     android:paddingStart="4dp"
    />
59.             </LinearLayout>
60.
61.
62.             <TextView
63.                 android:id="@+id/tvGenre"
64.
    android:layout_width="wrap_content"
65.
    android:layout_height="wrap_content"
66.                 android:layout_marginTop="2dp"
67.
    android:background="@drawable/badge_background"
68.                 android:paddingHorizontal="8dp"
69.                 android:paddingVertical="2dp"
70.
    android:textColor="@color/accent_color"
71.                 android:textSize="11sp" />
72.
73.
74.             <LinearLayout
```

```
75.
    android:layout_width="match_parent"
76.
    android:layout_height="wrap_content"
77.
    android:orientation="horizontal"
78.                 android:layout_marginTop="4dp">
79.
80.                 <TextView
81.                     android:id="@+id/tvRating"
82.                     android:layout_width="0dp"
83.
    android:layout_height="wrap_content"
84.                     android:layout_weight="1"
85.
    android:textColor="@color/rating_color"
86.                     android:textSize="12sp"
87.                     android:textStyle="bold" />
88.
89.                 <TextView
90.
    android:id="@+id/tvEpisodes"
91.
    android:layout_width="wrap_content"
92.
    android:layout_height="wrap_content"
93.
    android:textColor="@color/text_secondary"
94.                     android:textSize="11sp" />
95.             </LinearLayout>
96.
97.
```

```
98.             <TextView
99.                 android:id="@+id/tvPlot"
100.
    android:layout_width="match_parent"
101.
    android:layout_height="wrap_content"
102.                 android:layout_marginTop="4dp"
103.                 android:maxLines="3"
104.                 android:ellipsize="end"
105.
    android:textColor="@color/text_secondary"
106.                 android:textSize="12sp" />
107.
108.
109.             <LinearLayout
110.
    android:layout_width="match_parent"
111.
    android:layout_height="wrap_content"
112.
    android:orientation="horizontal"
113.                 android:layout_marginTop="8dp"
114.                 android:gravity="start">
115.
116.
    <com.google.android.material.button.MaterialBut
    ton
117.                     android:id="@+id/btnImdb"
118.
    android:layout_width="wrap_content"
119.
    android:layout_height="36dp"
```

```
120.
    android:text="@string/btn_imdb"
121.                     android:textSize="11sp"
122.                     app:cornerRadius="18dp"
123.
    style="@style/Widget.MaterialComponents.Button.
    OutlinedButton"
124.
    app:strokeColor="@color/accent_color"
125.
    android:textColor="@color/accent_color"
126.
    android:layout_marginEnd="8dp" />
127.
128.
    <com.google.android.material.button.MaterialBut
    ton
129.                     android:id="@+id/btnDetail"
130.
    android:layout_width="wrap_content"
131.
    android:layout_height="36dp"
132.
    android:text="@string/btn_detail"
133.                     android:textSize="11sp"
134.                     app:cornerRadius="18dp"
135.
    style="@style/Widget.MaterialComponents.Button"
136.
    app:backgroundTint="@color/accent_color" />
137.
138.             </LinearLayout>
139.         </LinearLayout>
140.     </LinearLayout>
```

```
141.
142.  </com.google.android.material.card.MaterialCardView>
```

Tabel 16. Source Code fragment_home.xml

```
1.   <?xml version="1.0" encoding="utf-8"?>
2.   <androidx.core.widget.NestedScrollView
3.       xmlns:android="http://schemas.android.com/apk/res/android"
4.       xmlns:app="http://schemas.android.com/apk/res-auto"
5.       android:layout_width="match_parent"
6.       android:layout_height="match_parent"
7.       android:background="@color/background_dark"
8.       android:fillViewport="true">
9.
10.      <LinearLayout
11.          android:layout_width="match_parent"
12.          android:layout_height="wrap_content"
13.          android:orientation="vertical"
14.          android:padding="12dp">
15.
16.
17.          <TextView
18.              android:layout_width="match_parent"
19.              android:layout_height="wrap_content"
20.              android:text="@string/label_highlights"
21.              android:textColor="@color/text_primary"
```

```
22.             android:textSize="18sp"
23.             android:textStyle="bold"
24.             android:layout_marginBottom="8dp"
    />
25.
26.
27.         <androidx.viewpager2.widget.ViewPager2
28.             android:id="@+id/viewPagerCarousel"
29.             android:layout_width="match_parent"
30.             android:layout_height="220dp"
31.             android:layout_marginBottom="4dp"
    />
32.
33.
34.
    <com.tbuonomo.viewpagerdotsindicator.DotsIndica
    tor
35.             android:id="@+id/dotsIndicator"
36.             android:layout_width="wrap_content"
37.
    android:layout_height="wrap_content"
38.
    android:layout_gravity="center_horizontal"
39.             android:layout_marginBottom="16dp"
40.             app:dotsColor="@color/accent_color"
41.             app:dotsSize="8dp"
42.             app:dotsSpacing="6dp"
43.             app:dotsWidthFactor="2.5" />
44.
45.
46.         <TextView
```

```
            android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="@string/label_popular"
android:textColor="@color/text_primary"
            android:textSize="18sp"
            android:textStyle="bold"
            android:layout_marginBottom="8dp"
/>


<androidx.recyclerview.widget.RecyclerView
            android:id="@+id/rvAnime"
            android:layout_width="match_parent"
android:layout_height="wrap_content"
android:nestedScrollingEnabled="false"
            android:clipToPadding="false" />

    </LinearLayout>

</androidx.core.widget.NestedScrollView>
```

Tabel 17. Source Code fragment_detail.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView
xmlns:android="http://schemas.android.com/apk/res/android"
```

```
4.      xmlns:app="http://schemas.android.com/apk/res-auto"
5.          android:layout_width="match_parent"
6.          android:layout_height="match_parent"
7.      android:background="@color/background_dark">
8.
9.          <LinearLayout
10.             android:layout_width="match_parent"
11.             android:layout_height="wrap_content"
12.             android:orientation="vertical">
13.
14.             <ImageView
15.                 android:id="@+id/imgDetailPoster"
16.                 android:layout_width="match_parent"
17.                 android:layout_height="280dp"
18.                 android:scaleType="centerCrop"
19.     android:src="@drawable/ic_placeholder" />
20.
21.             <View
22.                 android:layout_width="match_parent"
23.                 android:layout_height="60dp"
24.                 android:layout_marginTop="-60dp"
25.     android:background="@drawable/gradient_bottom_fade" />
26.
27.     <com.google.android.material.card.MaterialCardView
```

```
28.             android:layout_width="match_parent"
29.
android:layout_height="wrap_content"
30.
android:layout_marginHorizontal="16dp"
31.             android:layout_marginTop="-24dp"
32.             android:layout_marginBottom="16dp"
33.
app:cardBackgroundColor="@color/card_background
"
34.             app:cardCornerRadius="20dp"
35.             app:cardElevation="8dp">
36.
37.             <LinearLayout
38.
android:layout_width="match_parent"
39.
android:layout_height="wrap_content"
40.                 android:orientation="vertical"
41.                 android:padding="20dp">
42.
43.                 <LinearLayout
44.
android:layout_width="match_parent"
45.
android:layout_height="wrap_content"
46.
android:orientation="horizontal"
47.
android:gravity="center_vertical">
48.
49.                     <TextView
```

```
50.
    android:id="@+id/tvDetailTitle"
51.
    android:layout_width="0dp"
52.
    android:layout_height="wrap_content"
53.
    android:layout_weight="1"
54.
    android:textColor="@color/text_primary"
55.                         android:textSize="22sp"
56.
    android:textStyle="bold" />
57.
58.                     <TextView
59.
    android:id="@+id/tvDetailYear"
60.
    android:layout_width="wrap_content"
61.
    android:layout_height="wrap_content"
62.
    android:background="@drawable/badge_background"
63.
    android:paddingHorizontal="10dp"
64.
    android:paddingVertical="4dp"
65.
    android:textColor="@color/accent_color"
66.                         android:textSize="14sp"
67.
    android:textStyle="bold" />
68.                 </LinearLayout>
69.
```

```
70.                 <TextView
71.
    android:id="@+id/tvDetailGenre"
72.
    android:layout_width="wrap_content"
73.
    android:layout_height="wrap_content"
74.
    android:layout_marginTop="8dp"
75.
    android:background="@drawable/badge_background"
76.
    android:paddingHorizontal="10dp"
77.
    android:paddingVertical="3dp"
78.
    android:textColor="@color/accent_color"
79.                     android:textSize="13sp" />
80.
81.                 <View
82.
    android:layout_width="match_parent"
83.                     android:layout_height="1dp"
84.
    android:layout_marginVertical="12dp"
85.
    android:background="#3BFFFFFF" />
86.
87.                 <LinearLayout
88.
    android:layout_width="match_parent"
89.
    android:layout_height="wrap_content"
```

```
90.
    android:orientation="horizontal"
91.
    android:gravity="center_vertical">
92.
93.                     <TextView
94.
    android:layout_width="wrap_content"
95.
    android:layout_height="wrap_content"
96.
    android:text="@string/label_rating"
97.
    android:textColor="@color/text_secondary"
98.                         android:textSize="13sp"
99.
    android:layout_marginEnd="8dp" />
100.
101.                    <TextView
102.
    android:id="@+id/tvDetailRating"
103.
    android:layout_width="wrap_content"
104.
    android:layout_height="wrap_content"
105.
    android:textColor="@color/rating_color"
106.                        android:textSize="16sp"
107.
    android:textStyle="bold" />
108.                </LinearLayout>
109.
110.                <LinearLayout
```

```
111.
android:layout_width="match_parent"
112.
android:layout_height="wrap_content"
113.
android:orientation="horizontal"
114.
android:gravity="center_vertical"
115.
android:layout_marginTop="8dp">
116.
117.                    <TextView
118.
android:layout_width="wrap_content"
119.
android:layout_height="wrap_content"
120.
android:text="@string/label_episodes"
121.
android:textColor="@color/text_secondary"
122.                        android:textSize="13sp"
123.
android:layout_marginEnd="8dp" />
124.
125.                    <TextView
126.
android:id="@+id/tvDetailEpisodes"
127.
android:layout_width="wrap_content"
128.
android:layout_height="wrap_content"
129.
android:textColor="@color/text_primary"
```

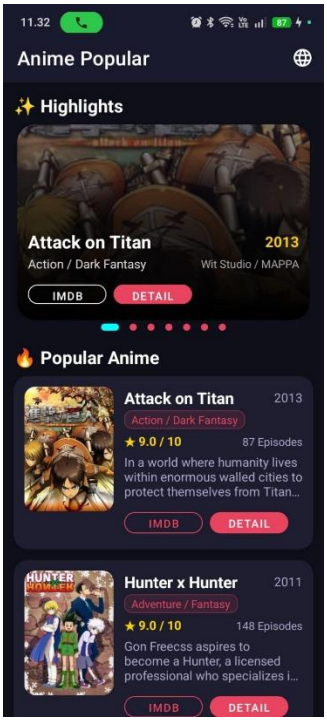
```
130.                            android:textSize="13sp"
    />
131.                </LinearLayout>
132.
133.                <LinearLayout
134.
    android:layout_width="match_parent"
135.
    android:layout_height="wrap_content"
136.
    android:orientation="horizontal"
137.
    android:gravity="center_vertical"
138.
    android:layout_marginTop="8dp">
139.
140.                    <TextView
141.
    android:layout_width="wrap_content"
142.
    android:layout_height="wrap_content"
143.
    android:text="@string/label_studio"
144.
    android:textColor="@color/text_secondary"
145.                            android:textSize="13sp"
146.
    android:layout_marginEnd="8dp" />
147.
148.                    <TextView
149.
    android:id="@+id/tvDetailStudio"
150.
    android:layout_width="wrap_content"
```

```
android:layout_height="wrap_content"
android:textColor="@color/text_primary"
                        android:textSize="13sp"
/>
                </LinearLayout>

                <View
android:layout_width="match_parent"
                    android:layout_height="1dp"
android:layout_marginVertical="12dp"
android:background="#3BFFFFFF" />

                <TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="@string/label_plot"
android:textColor="@color/text_secondary"
                    android:textSize="13sp"
                    android:textStyle="bold"
android:layout_marginBottom="6dp" />

                <TextView
```

```
172.
    android:id="@+id/tvDetailPlot"
173.
    android:layout_width="match_parent"
174.
    android:layout_height="wrap_content"
175.
    android:textColor="@color/text_primary"
176.                     android:textSize="14sp"
177.
    android:lineSpacingMultiplier="1.4" />
178.
179.
    <com.google.android.material.button.MaterialBut
    ton
180.
    android:id="@+id/btnDetailImdb"
181.
    android:layout_width="match_parent"
182.
    android:layout_height="48dp"
183.
    android:layout_marginTop="20dp"
184.
    android:text="@string/btn_open_imdb"
185.                     android:textSize="14sp"
186.                     app:cornerRadius="24dp"
187.
    app:backgroundTint="@color/imdb_yellow"
188.
    android:textColor="@android:color/black"
189.
    app:icon="@drawable/ic_open_external"
190.                     app:iconGravity="textStart"
```

```
191.  app:iconTint="@android:color/black" />
192.
193.              </LinearLayout>
194.  </com.google.android.material.card.MaterialCard
      View>
195.
196.      </LinearLayout>
197.
198.  </androidx.core.widget.NestedScrollView>
```

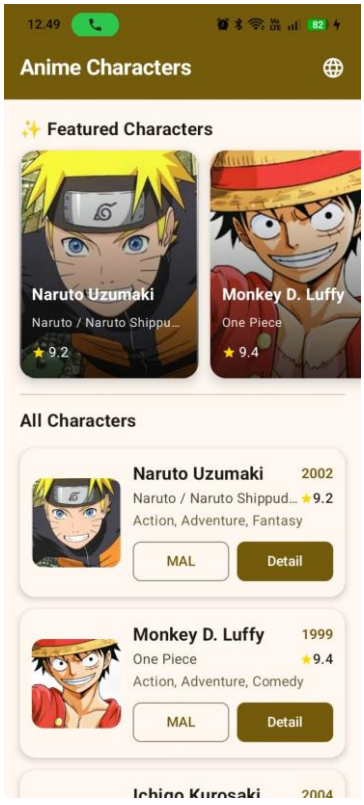
B. OUTPUT CODE



Gambar 1. Screenshot Hasil Jawaban Soal 1 XML

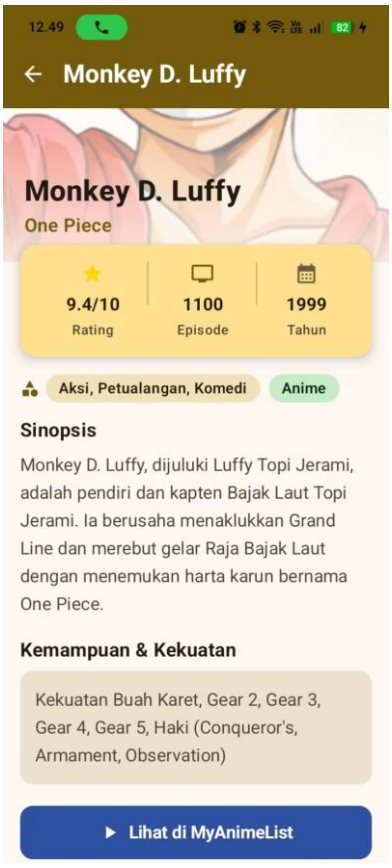
Gambar 2. Screenshot Hasil Jawaban Soal 1 XML

Gambar 3. Screenshot Hasil Jawaban Soal 1 XML



Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose

Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose

C. PEMBAHASAN

MainActivity.kt Compose

File MainActivity.kt pada antarmuka Compose menandakan bahwa tampilan akan digambar sepenuhnya oleh mesin Compose melalui blok fungsi “setContent”. Di dalam file ini, program mengambil data preferensi bahasa pengguna yang tersimpan (Shared Preferences) dan menerapkan konfigurasi lokal secara terpusat untuk mendukung pergantian fitur multi-bahasa sebelum menyuntikkan tata letak awal “Surface” dan menginisiasi navigasi rute lewat “NavGraph” dengan bantuan “rememberNavController”.

AnimeViewModel.kt Compose

File AnimeViewModel.kt menerapkan pola arsitektur pengelolaan state (status data) reaktif agar informasi list tidak hilang seketika saat orientasi layar gawai diputar atau berubah. Dengan memanfaatkan fungsi “MutableStateFlow” dan “StateFlow” dari pustaka coroutines Kotlin, kelas ini bertanggung jawab sebagai wadah pengamat untuk menyimpan status indeks rotasi karakter (Carousel) serta memegang referensi daftar array pangkalan data anime yang nantinya dipantau dan digambar secara otomatis oleh komponen antarmuka.

AnimeData.kt Compose

File AnimeData.kt difungsikan oleh program sebagai modul repositori penyedia objek statis atau data rekaan murni (dummy) yang memuat informasi terkait entitas daftar anime secara spesifik. Di dalamnya, rincian daftar anime disajikan dalam pola cetak wujud "AnimeCharacter" di mana masing-masing nilainya ditautkan secara langsung menggunakan ID Resource String (seperti R.string.char1_name) untuk menyederhanakan mekanisme transisi pemutakhiran translasi multibahasa (Indonesia/Inggris), yang disandingkan dengan parameter penautan URL gambar ilustrasi.

AnimeCharacter.kt Compose

File AnimeCharacter.kt merupakan data class spesifik bawaan Kotlin yang dikhususkan sebagai cetak biru model pendefinisian atribut bagi satu objek karakter anime. Elemen-elemen penampung dalam kelas ini dirancang kuat untuk mengatur tipe variabel bawaan dari informasi inti, seperti atribut "id" yang berbentuk Integer, ID referensi teks bahasa untuk nama, rating dengan tipe pecahan (Float), dan juga nilai genre agar mudah diproses ulang di tiap penataan baris menu secara persisten.

NavGraph.kt Compose

File NavGraph.kt dioperasikan selaku pusat sistem kontrol lalu lintas perpindahan rute dan pergantian antar-layar memanfaatkan infrastruktur perpustakaan kerangka navigasi bawaan “NavHost”. Logika struktur di file ini menyusun rute tujuan destinasi layar secara runtut, meliputi pemetaan pembuatan rute ke ruang detail berbasis operan indeks argumen angka ID agar tepat

mengekstrak objek tujuan ketika ditekan, serta mengakomodir rute transisi eksternal langsung menuju ruang pengaturan ganti bahasa aplikasi.

Screen.kt Compose

File Screen.kt menggunakan kelebihan konstruksi properti sealed class pada standar bahasa agar bisa mengunci dan merangkum ragam alamat string penamaan rute menjadi bagian statis yang aman dan tidak dapat diubah-ubah, di antaranya seperti “Home”, “Detail”, hingga “Language”. Tujuannya agar saat arsitektur layar membutuhkan pemanggilan di pengontrol NavGraph, program terhindar dari potensi kesalahan penghentian sistem yang konyol karena kekeliruan pengetikan (typo), terkhusus pada fungsionalitas rute mendetail yang melibatkan pembentukan string variabel lewat tambahan argumen.

MainActivity.kt XML

Dalam koridor pendekatan desain klasikal berbasis XML, MainActivity.kt berperan teguh menyokong struktur kerangka dasar Single-Activity yang membungkus semua tata letak transisi. Teknik "findViewById" diusangkan lalu digantikan dengan fungsional “ViewBinding” untuk mempertautkan tata letak dengan mudah, berlanjut menuju pembentukan sistem “NavHostFragment” guna menghubungkan manuver pergerakan fragmen yang berpadu dengan aksi kustom bilah pita (Toolbar) teratas serta merakit ulang pemuatan konfigurasi preferensi ragam penuturan bahasa bawaan secara periodik.

AnimeAdapter.kt XML

File AnimeAdapter.kt berfungsi penuh mencetak dan menggandakan skema barisan item RecyclerView tanpa batas dengan mengeksekusi kelas perpanjangan dari standar “ListAdapter” yang diperkuat kemampuan DiffCallback agar perbandingan dan pembaruan elemen baris tampak natural nan mulus tanpa perlu merender dari ulang. Memasuki fase “ViewHolder”, ia mentransformasi logika pembacaan variabel objek lalu menerjemahkannya untuk menyuntikkan data ke teks grafis sembari mendelegasikan pustaka tangguh (Glide) dalam merender URL jaring internet menjadi poster serta merekatkan event klik di tombol eksternal maupun internal navigasi.

AnimeRepository.kt XML

File AnimeRepository.kt adalah file terisolasi khusus wadah penyimpanan gudang data statis (dummy data) yang bertugas mereturn nilai kembalian kolektif untuk dikonsumsi. Tanpa memberatkan muatan tampilan visual, sistem pembentukan objek disusun mengandalkan wujud kelas data secara eksplisit dengan menyimpan masing-masing atributnya menjadi sepasang salinan, contohnya pada narasi sinopsis ("plotEn" dan "plotId") yang disematkan secara spesifik agar fragmen detail UI layer dengan mudah memilihnya berdasar logika pengaturan bahasa yang dicari.

AnimeItem.kt XML

File AnimeItem.kt hanya ditugaskan sebagai pondasi abstraksi dan wujud dari kerangka arsitektur struktur pangkalan objek dengan jenis data class. Modifikasi paling mendasar di file kerangka model ini adalah pembungkusan lewat alat dekorator khusus "@Parcelize" lalu mengadopsi pewarisan "Parcelable", di mana tindakan fungsional ini krusial di Android agar mesin sanggup menggulung, memadatkan bita, hingga mengemas semua serpihan informasi anime utuh secara efisien via argumen SafeArgs tanpa kerusakan korupsi struktur saat menyeberang fragment layar.

HomeFragment.kt XML

File HomeFragment.kt menempati pos krusial berwujud presentasi wajah utama menu aplikasi bagi para pengakses yang dijalin dari hasil ikatan ViewBinding. Lewat intervensi pada fungsi bawaan sistem perputaran gawai, kelas mendeteksi orientasi gawai aktual untuk otomatis menerapkan tampilan deret linear memanjang (LinearLayoutManager) pada potret, atau susunan bertumpuk dua kolom via pembagian GridLayout pada layar mendatar, dan ditutup lewat pembentukan tautan pemanggil fitur Carousel hingga pengaturan delegasi klik implisit (intent URL) yang interaktif.

DetailFragment.kt XML

Pada file DetailFragment.kt, kendali presentasi layar berpusat di ruang penerimaan argumentasi di mana objek "Parcelable" disedot sepenuhnya melalui bantuan keping perantara. Mekanisme internal mencari tahu rujukan riwayat penyetelan Shared Preferences yang pernah diinisiasi untuk mengembalikan teks informasi yang tepat, mengatur pelabelan properti teks, dan menjahitnya kembali ke dalam elemen tampilan antarmuka beserta implementasi peluncur gambar instan ke wadah kanvas yang tersedia.

AnimeViewModel.kt XML

File AnimeViewModel.kt untuk perancangan variasi arsitektur XML memegang mandat menyuplai deretan variabel referensi pasif yang dikoleksi eksklusif dari pusat gudang statis (Repository). Menyatu di dalam kerangka kerja ViewModel, daftar tersebut diselimuti pembungkus sistem asinkronisasi yang menjembatani lapisan fragmen dan data layer agar program terbebas dari kesalahan cacat fatal karena kehilangan alur memori dan mencegah terhapusnya daftar item saat pengguna gawai menggulir atau mengubah proporsi fisik ukuran secara dinamis.

activity_main.xml

File activity_main.xml berfungsi selayaknya denah dasar rumah atau kerangka terluar untuk menampung lalu lalang komponen transisi fragment di kerangka sistem Single-Activity. Disusun hanya dengan kode penanda (XML) yang statis, antarmuka utamanya mengandalkan satu tag "NavHostFragment" yang melengkupi sebagian besar luas ruang layar dengan sisipan dekorasi bilah kepala di

bagian paling pucuk (ActionBar) yang memastikan integrasi reaktif antara menu antarmuka dan panel layar di dalamnya.

item_anime.xml

File item_anime.xml diplot merujuk pada tata letak kardus bingkai dari komponen satuan terkecil per baris tampilan list aplikasi list tanpa gulir yang berulang-ulang. Agar terlihat kekinian ala rancangan sistem Material Design, komponen "CardView" membungkus struktur guna menghasilkan rekayasa pangkasan tepi lengkung (rounded corners), seraya menyusun kotak muatan bagi rupa poster, paragraf pendeskripsi identitas, hingganya peletakan sepasang susunan instrumen tombol yang mendistribusikan tanggapan klik.

fragment_home.xml

File fragment_home.xml merupakan kanvas sketsa statik perwujudan kombinasi instrumen ruang tatap muka interaktif pertama yang berpusat pada halaman Beranda utama. Desain menyematkan satu deret tag komponen fungsi Carousel mengusung “ViewPager2” bersanding penanda titik (dots indicator) di titik atas untuk disisipkan, sedangkan bidang bawah menampung ruang gulir “RecyclerView” memanjang secara proporsional supaya bisa memaparkan barisan komponen detail anime berjumlah banyak yang padu bersinkronasi dari tumpukan elemennya.

fragment_detail.xml

File fragment_detail.xml dibangun dengan menugasi area pinggiran antarmuka layar dengan komponen lentur pelindung guliran (NestedScrollView) demi memastikan rentetan rincian teks sinopsis naratif mendalam mampu dibaca hingga beres merespons usapan layar gawai pengguna. Sorotan pilar tampilan dibentangkan lewat kanvas visual poster utuh yang membentang disusul penyusunan bertumpuk untuk setiap properti statis keterangan label detail mulai judul hingga ulasan.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. PEMBAHASAN

Meskipun kehadiran LazyColumn pada Jetpack Compose menawarkan penulisan kode yang jauh lebih ringkas, RecyclerView berbasis XML tetap dipertahankan karena perannya yang sangat vital dalam memelihara aplikasi berskala besar (legacy). Banyak perusahaan dengan basis kode Android lama enggan melakukan migrasi total ke Compose karena proses refactoring menyeluruh akan memakan waktu yang lama, membutuhkan biaya besar, dan berisiko mengganggu stabilitas sistem yang sudah berjalan dengan baik. Oleh karena itu, penguasaan terhadap komponen lama ini tetap menjadi tuntutan industri yang tidak bisa dihindari oleh pengembang.

Di sisi lain, RecyclerView menawarkan tingkat kustomisasi yang lebih mendalam sekaligus menjadi sarana pembelajaran fondasi sistem Android yang esensial. Secara teknis, komponen ini memberikan kontrol penuh kepada pengembang untuk merancang animasi transisi yang rumit, tata letak asimetris, hingga jarak antar-item yang spesifik melalui manipulasi ItemAnimator, ItemDecoration, dan LayoutManager. Lebih jauh lagi, keterlibatan pola Adapter dan ViewHolder di dalamnya secara langsung melatih pengembang untuk menguasai mekanisme daur ulang memori perangkat secara efisien, yang merupakan keterampilan fundamental dalam mengoptimalkan performa antarmuka (UI) aplikasi seluler.

SOAL 3

Pada desain arsitektur kotlin, terdapat beberapa arsitektur dengan salah satunya adalah CLEAN ARCHITECTURE, menurut pendapat kalian, mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project?

A. PEMBAHASAN

Menurut saya, penerapkan Clean Architecture itu sangat penting. Apalagi untuk proyek aplikasi berskala besar. Dengan Clean Architecture, seperti sedang menyusun kode-kode tersebut ke dalam rak atau lapisannya masing-masing. Ada lapisan yang khusus mengurus data saja (Data Layer), logika aplikasinya (Domain Layer), dan tampilan antarmukanya (Presentation Layer). Karena setiap tugas dipisah-pisah seperti ini, nantinya kalau ada error, kita bisa langsung mencari letak kerusakannya di folder yang spesifik tanpa harus membongkar seluruh kode dari awal. Terlebih lagi, ini membuat proyek aplikasi kita bisa dikerjakan oleh beberapa orang sekaligus (teamwork) tanpa takut kode saling bertabrakan (bentrok), sehingga aplikasi jadi jauh lebih rapi, terstruktur, dan gampang dites (testable).

TAUTAN GIT

https://github.com/ERISCHA24/PRAKTIKUMMOBILE.git